

AI Research Assistant:

1.)Project Overview:

The objective of this project is to design and develop an **AI Research Assistant** that enables users to interact with research papers through natural language. The system is built using a Retrieval-Augmented Generation (RAG) architecture, combining semantic document retrieval with the reasoning capabilities of Large Language Models (LLMs). Rather than functioning as a conventional document search engine, the proposed system constructs a searchable semantic knowledge base from uploaded research papers, allowing users to efficiently explore, understand, and analyze technical literature.

The application accepts research papers in PDF format and processes them through a document ingestion pipeline. During this process, document contents are extracted, enriched with structured metadata, divided into semantically meaningful chunks, converted into dense vector embeddings, and indexed within a vector database. This indexed knowledge base serves as the foundation for all intelligent services provided by the application.

Using this knowledge base, the system supports multiple AI-assisted functionalities including context-aware question answering, semantic document search, automatic document summarization, and quiz generation for self-assessment. By retrieving relevant document segments before invoking the language model, the system ensures that generated responses remain grounded in the uploaded research material, improving both factual accuracy and transparency.

The proposed architecture is modular and scalable, allowing additional capabilities such as multi-document reasoning, citation-aware responses, advanced retrieval strategies, and personalized document management to be incorporated with minimal modifications to the core system.

2.)Project Objectives:

- Develop an automated document ingestion pipeline for research papers in PDF format.
- Construct a semantic knowledge base using vector embeddings and a vector database.
- Implement metadata enrichment to support efficient document organization, filtering, and citation-aware retrieval.
- Enable context-aware question answering using Retrieval-Augmented Generation.
- Generate concise summaries of uploaded research papers.
- Automatically generate quizzes to assist users in understanding technical material.
- Design a modular software architecture that supports future extensions and additional AI services.

[Type here]

- Provide an intuitive interface for interacting with uploaded research documents.

3.)System Requirements:

3.1.) Functional Requirements:

The AI Research Assistant is designed to provide an intuitive platform for interacting with research papers using Retrieval-Augmented Generation (RAG). The system shall support the following functional requirements.

3.1.1.) **Document Upload**: The system shall allow users to upload research papers in PDF format through the user interface. Uploaded documents will serve as the primary source of information for subsequent processing and retrieval tasks.

3.1.2.) **Document Processing**: The system shall extract textual content from uploaded PDF documents using a document loader. Each document shall be represented internally as a collection of LangChain **Document** objects containing the extracted text and associated metadata.

3.1.3.) **Metadata Enrichment**: The system shall enrich the extracted documents with additional metadata before indexing. This metadata may include information such as document title, source filename, page number, document identifier, and other relevant attributes that facilitate efficient organization and retrieval.

3.1.4.) **Document Chunking**: The system shall divide large documents into smaller overlapping text chunks using a recursive character-based text splitter. Chunking ensures that each segment remains within the context window of the embedding model while preserving semantic continuity.

3.1.5.) **Embedding Generation**: The system shall convert each document chunk into a dense vector representation using a HuggingFace sentence embedding model. These embeddings capture the semantic meaning of the text and enable similarity-based retrieval.

3.1.6.) **Knowledge Based Construction**: The system shall construct a searchable semantic knowledge base by storing document chunks, their corresponding embeddings, and metadata within a Chroma vector database.

3.1.7.) **Semantic Retrieval**: The system shall retrieve the most relevant document chunks in response to a user's natural language query by performing similarity search on the indexed knowledge base.

3.1.8.) **Context Aware Question Answering**: The system shall provide answers to user queries by supplying the retrieved document context to a Large Language Model. Responses shall be generated based on the retrieved information rather than solely on the model's internal knowledge.

[Type here]

3.1.9.) **Document Summarization:** The system shall generate concise summaries of uploaded research papers to help users quickly understand the key concepts and findings contained within the documents.

3.1.10.) **Quiz Generation:** The system shall generate question sets based on the uploaded research papers to assist users in evaluating their understanding of the document content.

3.2.) Software Requirements:

Component	Technology
Programming Language	Python
User Interface	Streamlit
Backend Framework	FastAPI
LLM Framework	Langchain
Embedding Model	HuggingFace Sentence Transformers
Vector Database	Chroma/FAISS
Large Language Model	Groq (Llama Model 3.3-70b-versatile)
PDF Processing	PyPDF Loader
Environment Management	Python dotenv file

4.)System Architecture and Design:

4.1.) **Architectural Overview:** The proposed AI Research Assistant follows a modular architecture based on the Retrieval-Augmented Generation (RAG) paradigm. The system is designed by separating the complete document processing and retrieval pipeline into independent functional modules, each responsible for a specific stage of the workflow. This modular design improves maintainability and extensibility by ensuring that changes within one module have minimal impact on the remaining components of the system.

The overall architecture consists of five primary modules:

- Document Ingestion Module
- Metadata Enrichment Module
- Knowledge Base Construction Module
- Retrieval Module
- AI Service Module

Each module communicates through well-defined data objects, enabling the system to process uploaded research papers, construct a semantic knowledge base, retrieve relevant information, and generate intelligent responses.

The high-level architecture of the proposed system is illustrated below.

[Type here]

4.2.) Document Ingestion Module:

The Document Ingestion Module serves as the entry point of the system. Its responsibility is to accept uploaded research papers and convert them into an internal representation suitable for further processing.

The module utilizes LangChain's "PyPDFLoader" to extract textual information from uploaded PDF documents. Rather than treating the document as a single block of text, each page is represented as an individual "Document" object.

Each "Document" object contains two primary components:

- "page_content" = The textual content extracted from the document.
- "metadata" = descriptive information associated with the extracted text, such as the source document and page number stored as a python dictionary.

Representing document in this structured format simplifies subsequent operations including metadata enrichment, chunking and retrieval.

Input = Research paper in PDF format

Output = Python List of Langchain "Document" objects

4.3.) Metadata Enrichment Module: The Metadata Enrichment Module enhances the extracted document objects with additional structured information before they are indexed into the knowledge base.

While the document loader automatically provides basic metadata such as source filename and page number, this module introduces application-specific metadata that improves document organization and enables efficient retrieval.

Each document is associated with a metadata dictionary that stores descriptive information about the source document. The metadata is propagated to every chunk generated from the document, ensuring that contextual information remains available throughout the retrieval process.

Typical metadata maintained by the system includes:

- "paper_id" = Unique Identifier for the uploaded paper
- "source" = Original Filename
- "page" = Page Number
- "title" = Document Title (if available)

4.4.) Knowledge Based Construction:

The Knowledge Base Construction Module transforms enriched documents into a searchable semantic repository.

[Type here]

This module performs document chunking, embedding generation, and vector indexing to construct the Chroma knowledge base.

Large research papers often exceed the maximum context length supported by embedding models and Large Language Models. To overcome this limitation, each document is divided into smaller overlapping chunks using the Semantic Base Chunking.

Once chunking is complete, every chunk is converted into a dense vector embedding using a HuggingFace sentence embedding model. The embedding captures the semantic meaning of the text, allowing conceptually similar passages to be identified even when they use different wording.

The generated embeddings, along with the chunk text and associated metadata, are then indexed into the Chroma vector database.

Input = Enriched Document Objects

Output = Indexed Semantic Knowledge Base.

4.5.) Retrieval Module:

The Retrieval Module is responsible for locating the most relevant information within the knowledge base in response to user queries.

When a user submits a natural language query, the query is converted into an embedding using the same embedding model employed during knowledge base construction. Maintaining the same embedding space for both documents and queries ensures meaningful similarity comparisons.

The query embedding is compared against the embeddings stored within the Chroma vector database. Based on vector similarity, the retriever identifies the document chunks that are most relevant to the user's request.

The retrieved chunks are then forwarded to the AI Service Module as contextual information for response generation.

4.6.) AI Service Module:

The AI Service Module generates intelligent outputs using the contextual information retrieved from the knowledge base.

Instead of relying solely on the pretrained knowledge of the language model, the AI Service Module receives both the user's query and the retrieved document chunks. This contextual information is incorporated into the prompt provided to the Groq-hosted Large Language Model, enabling responses that are grounded in the uploaded research papers.

The same retrieval pipeline supports multiple application features, including:

[Type here]

- Context aware question answering
- Document Summarization
- Quiz Summarization

By sharing a common retrieval mechanism, the system avoids redundant document processing while ensuring consistency across all AI-assisted functionalities.

4.7.) Module Interaction:

The interaction among the modules is sequential and forms a complete document processing pipeline.

- 1.) The Document Ingestion Module extracts text from uploaded PDF documents.
- 2.) The Metadata Enrichment Module augments the extracted documents with structured metadata.
- 3.) The Knowledge Base Construction Module chunks the documents, generates embeddings, and indexes them within the Chroma vector database.
- 4.) The Retrieval Module searches the knowledge base to identify document chunks relevant to the user's query.
- 5.) The AI Service Module uses the retrieved context to generate the final response through the Large Language Model.

This separation of responsibilities results in a modular architecture that is easier to maintain, extend, and debug.

4.8.) Application Data Management Module:

Purpose:

This module manages all persistent information required by the application that is unrelated to semantic retrieval.

It stores:

- Research Projects
- Uploaded Paper Records
- Chat Sessions
- Conversational History

The original PDF documents remain stored in the local file system, while the database stores references (file paths) to those documents.

Database Schema:

Attribute	Description
Project_ID	Unique Identifier
Project Name	Project Title
Created_at	Creation timestamp

[Type here]

This was the table schema for ‘Projects’

‘Papers’ table:

Attributes	Description
Paper ID	Unique Paper Identifier
Project ID	Parent Project
Title	Paper title
File Path	Location of the PDF file

‘Chats’ table:

Attributes	Description
Chat ID	Unique Conversation
Project ID	Parent Project
Created at	Creation TimeStamp

‘Messages’ table:

Attributes	Description
Message ID	Unique Message
Chat ID	Conversation Identifier
Role	User/Assistant
Message	Message Text
Timestamp	Time Generated

The proposed architecture separates application data from semantic knowledge. Application-level information, including research projects, uploaded document records, chat sessions, and conversation history, is managed using a relational database (SQLite for local deployment or PostgreSQL for scalable deployment). Original PDF documents are stored in the local file system, while processed document chunks, embeddings, and associated metadata are maintained within the Chroma vector database for efficient semantic retrieval.